AI Mahjong Commentator

Team 6 · Youlin Qu · Lenghan Zhu · Rongcheng Guo · Pingfan Bu

The Concept | The Approach | The Assumptions | The Dataset | The Results

What We Built

THE PROBLEM

- **Online matches feel silent.**
  Without a commentator, a winning tenpai hand looks the same as an ordinary discard to any spectator.
- **New players can't follow strategy.**
  Mahjong has 30–80 moves per hand with invisible hand states — beginners have no signal about what matters.
- **Spectating without a streamer is passive.**
  No narration means no emotional arc, no tension — viewers drop off.
- **Human commentators are expensive to scale.**
  Studios can't afford a commentator for every room, every match, around the clock.

Matches are hard to follow and easy to quit.

→

THE SOLUTION

- **An AI commentator that reacts to live events.**
  Every discard, peng, gang, and win triggers a commentary sentence in under one second.
- **Context-aware, not just move-aware.**
  The AI tracks the full game arc — not just the current tile — producing commentary with strategic depth.
- **Bilingual: English and Chinese.**
  Language toggled live — same game event generates commentary in EN or ZH on the fly.
- **Plugs in without touching game code.**
  Connects to any game server that exposes structured WebSocket events — no engine modification.

Built and live: connected to a real 4-player Mahjong server with audio TTS output.

How the Pipeline Works

1

Game Server
WEBSOCKET EVENTS

Live Mahjong server pushes structured JSON events on every player action.

Event types:
· playCard — tile discarded
· peng / gang — claim or reveal
· win — hand declared

Full hand arrays included — no screen capture, no vision.

→

2

Context Assembly
3-LAYER MEMORY

Each event is parsed and enriched before the LLM sees it.

· Hand-strength (shanten) computed in < 5ms
· Player name resolved from room state
· AI narrative + last 10 moves + current detail assembled into a ~370-token prompt (design estimate)

Prompt size stays constant — move 80 costs the same as move 8.

→

3

Dual-Agent LLM
COMMENTATOR + SUMMARIZER

🎙️ Commentator — per event, < 800ms target. Picks strategic angle A or B per move.

📊 Summarizer — every 8 events, async background. Writes a 2–3 sentence narrative from full history.

Both powered by Gemini. Commentary prompted in EN or ZH based on viewer setting.

→

4

Browser UI
REACT + WEBSOCKET

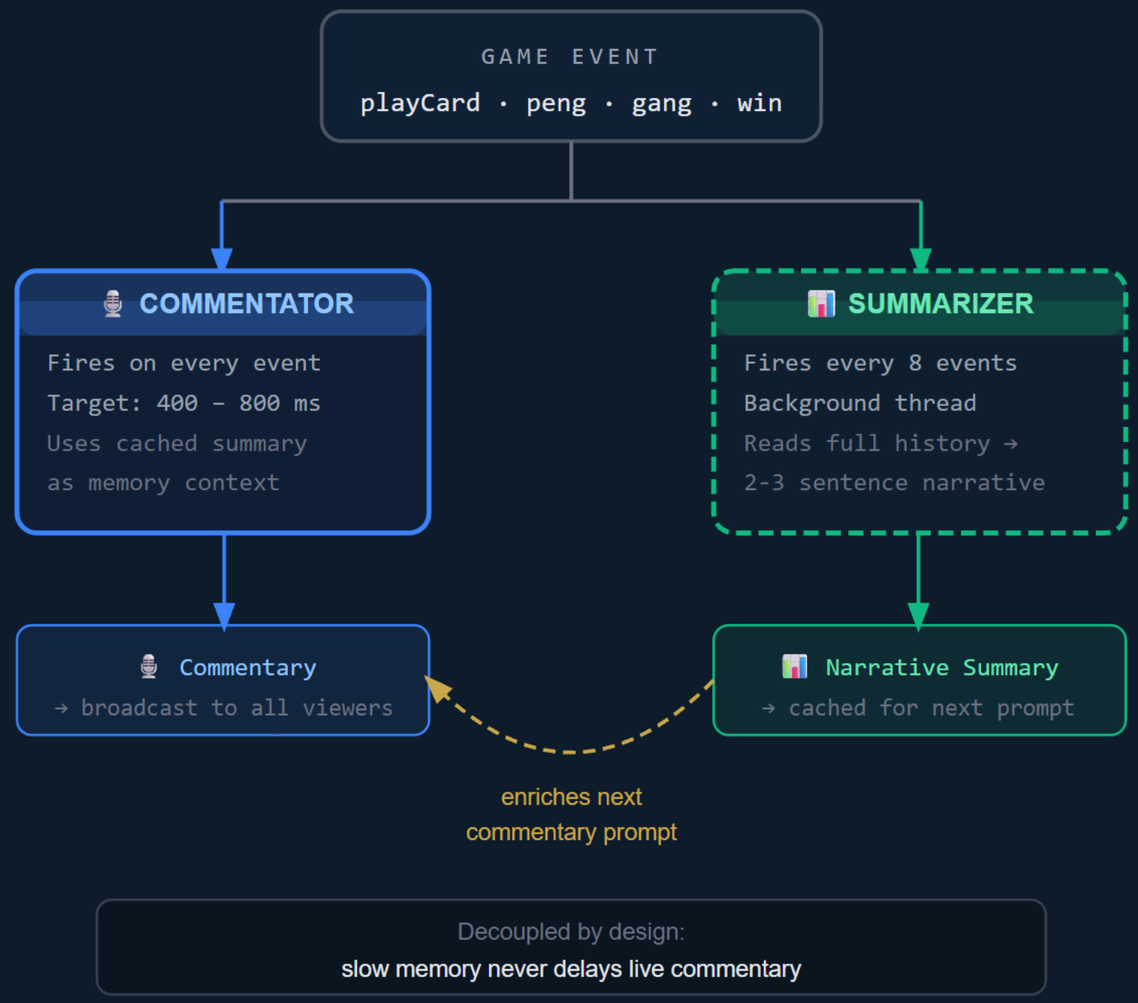
React web app — any viewer joins a room by URL.

· Live commentary feed, scrolling
· Optional TTS audio (OpenAI) for spoken commentary
· Debug sidebar: player hands, shanten live, event trace
· New clients get last 50 messages instantly on join

Multiple rooms run concurrently, each fully isolated.

Mahjong is the proof-of-concept — any game exposing structured WebSocket events can slot into this pipeline.

Two Agents, One Live Stream



THE CHALLENGE

Commentary must arrive in **under one second**. But building a sense of how the game has evolved takes time. A single agent cannot serve both masters.

COMMENTATOR

Strict real-time budget. Uses the *latest cached summary* as long-term context — never waits for a new one. 400–800 ms target.

SUMMARIZER

No deadline. Reads the *full game history* and writes a narrative covering player trends, turning points, and key dynamics. Runs in the background.

Validated by Microsoft Real-Time Chess Commentary — *"chaining prior outputs with new inputs forms a coherent narrative."*

3-Layer Memory: Solving the Stateless LLM Problem

LAYER 1 · LONG-TERM MEMORY ~60 tokens

AI-written narrative of ALL previous moves

Generated by the background Summarizer agent every 8 moves.

"Player 2 has been building towards a pure-suit hand since move 12. Player 3's aggressive peng at move 31 changed…"

LAYER 2 · RECENT EVENTS (last 10 moves) ~150 tokens

Structured facts: who did what, how close to winning

```
#41 Player 1 discards 3-man | 2 from tenpai | 18 tiles left
#42 Player 2 calls peng | Tenpai | 17 tiles left
#43 Player 3 discards 9-char | 3 from tenpai | 16 tiles left …
```

Hand-strength number computed by algorithm — not guessed by AI

LAYER 3 · THIS EXACT MOMENT ~80 tokens

Full tile detail: exact hand, open sets, distance to win

```
Player 3 throws 9-char.
Hand: 1-man 2-man 3-man | 5-dot 6-dot (waiting 4 or 7-dot) — Tenpai.
Open melds: peng 3-char 3-char 3-char.
```

↓

🤖 **LLM generates commentary**

Total: ~370 tokens (design estimate) — fixed, regardless of game length

THE PROBLEM

LLMs are stateless. By move 60, the full game history is thousands of tokens — too expensive to send every time, and still growing.

OUR SOLUTION

Three layers with different granularity: **AI-compressed history** covers everything before, **a sliding window** covers what's happening recently, and **full detail** covers this exact moment.

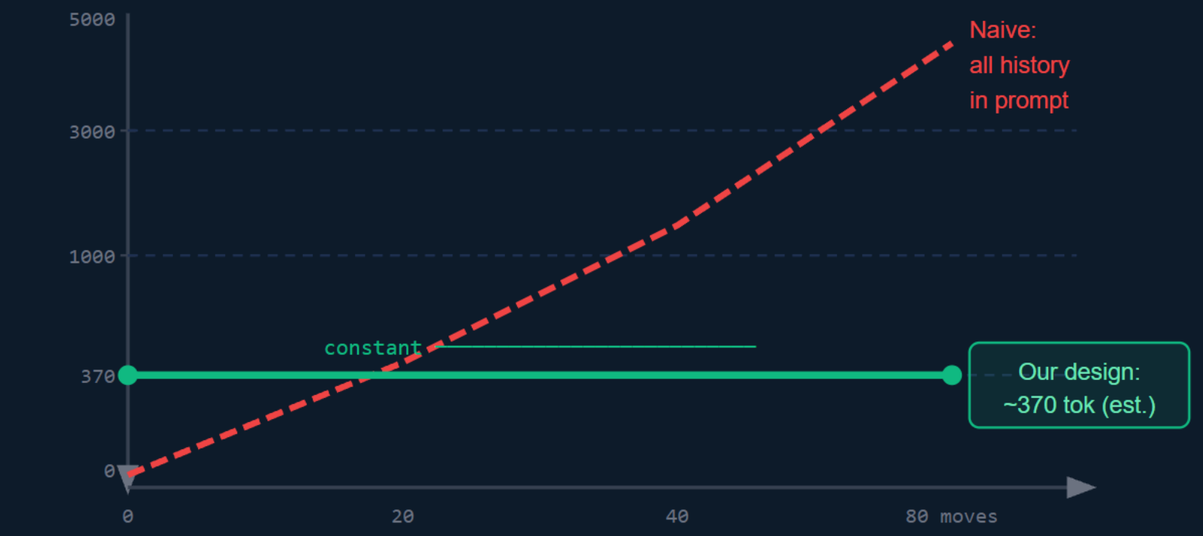
THE KEY WIN

The 80th move costs **exactly the same** as the 8th. Prompt size is constant — so latency is constant. No degradation as the game progresses.

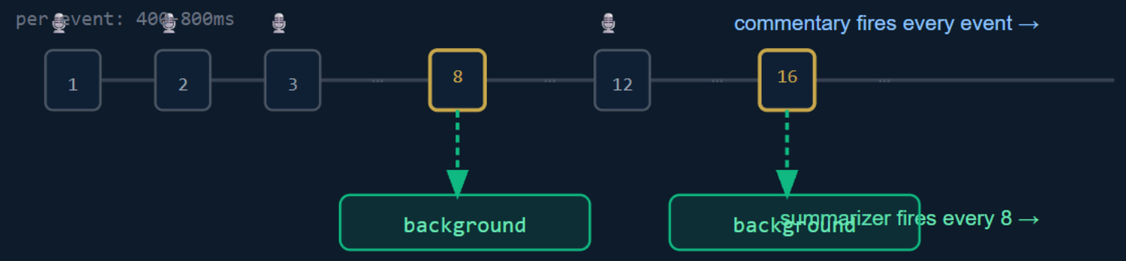
"Structured domain features — not subjective labels — reduce hallucination and improve commentary relevance." — ArXiv 2024, Concept-Guided Chess Commentary

Engineering Choices That Keep Latency Stable

PROMPT TOKEN COST AS GAME PROGRESSES



ASYNC NON-BLOCKING ARCHITECTURE



The two paths never block each other. Commentary keeps flowing even while a new summary is being written.

Full recompute, not incremental:

Each summary is written fresh from raw event data — errors never accumulate across summaries.
Validated: DEV.to async agents — "if LLM unavailable, messages queue; processing resumes automatically."

✓ NON-BLOCKING ASYNC

Summarization never stalls live commentary. Commentary keeps flowing; the new summary silently becomes available for the next call.

✓ CONSTANT TOKEN BUDGET

~370 tokens (design estimate). The 80th move costs the same as the 8th. Prompt cost doesn't grow with game length — latency stays stable.

✓ FRESH START EVERY SUMMARY

Full recompute from raw event data — not incremental updates. One wrong summary can't poison the next. Quality stays consistent throughout the game.

✓ OBJECTIVE SIGNALS ONLY

Hand strength is computed by a shanten algorithm — *not guessed by the LLM*. Feeds every move record. Reduces hallucination, grounded in ArXiv 2024 chess findings.

Key Design Assumptions

USER

Audience Has Diverse Skill Levels

Commentary must serve beginners (who need context), casual viewers (who want drama), and experienced players (who want strategy). A single style won't serve all three.

The LLM is prompted to *pick the more insightful angle* per move — strategic intent or hand shape opportunity — letting it adapt depth naturally to the game situation rather than requiring us to categorise viewers.

In practice: same event, same prompt — LLM naturally varies depth.

GAME

Game State Is Fully Observable

The system assumes the game server exposes structured event data — tile IDs, player hand arrays, peng/gang sets, and remaining tile count — over a real-time connection.

Our Mahjong integration uses a WebSocket connection to the server. Full hand state is available server-side — no screen scraping, no vision model, no inference about hidden tiles.

In practice: all four players' hands are known — shanten computed exactly.

SCALE

Commentary Generalises Beyond Mahjong

The event pipeline, context assembly, and dual-agent design are game-agnostic. Only the prompt system instructions and tile-name utilities are Mahjong-specific.

The assumption is that other card and tabletop games expose similar structured event streams. The RoomManager architecture supports multiple concurrent rooms — scaling to many games in parallel is a configuration change, not a rewrite.

In practice: swap prompt + tile utility → adapt to a new game.

COST

No Heavy Live-Ops Overhead

Unlike human commentators, the AI runs at low marginal cost per session. Studios can enable commentary for every room, every match — not just high-profile ones — without adding headcount.

The target ~370-token prompt (design estimate) keeps per-call cost stable regardless of game length. The async Summarizer uses a background call that doesn't affect real-time latency. Gemini Flash Lite keeps commentary costs per match low.

In practice: ~370 tokens per event call (design estimate), independent of game length.

What Data Drives the System

INPUT DATA

Game Events

WebSocket JSON events pushed on every player action: playCard, peng, gang, win, startGame, roomState. Each event carries the acting player's ID and the tiles involved. These are the triggers — the system fires on every key event, automatically.

Player Hand State

Per-player tile arrays for handCards[], pengCards[], gangCards[] — all four players' full hands visible server-side. Used to compute the shanten number (distance to winning) by backtracking algorithm in < 5ms. Not estimated — exact.

Game State

Remaining tiles in the wall, total move count, room player list with display names. Tracks game progression and time pressure — fed into every structured fact so the LLM knows how late in the game each move occurs.

Full Event History

Every move stored locally from game start as compressed structured facts (~15 tokens each). Passed to the background Summarizer in full — not sampled — to generate a complete narrative of player trends and turning points. An 80-move game is ~1200 tokens of raw material for the Summarizer.

HOW IT'S USED

① **Event → Structured Fact.** Player name resolved, shanten computed in < 5ms, action described in natural language. Appended to the complete event history log.

② **3-Layer Prompt Assembled.** AI narrative summary (background Summarizer) + last 10 moves as structured facts + current action with full tile detail. Total is ~370 tokens by design estimate, regardless of game length.

③ **Commentary Broadcast.** LLM response delivered simultaneously to all connected browser clients via WebSocket. Optionally spoken aloud by OpenAI TTS, queued to play one sentence at a time.

④ **Background Summary Refresh.** Every 8 events the Summarizer reads the full event history and rewrites the narrative from scratch — no incremental drift — and silently makes it available for the next commentary call.

What We Achieved

< 800ms

End-to-End Live System

- **Live connection** to a 4-player Mahjong game server via WebSocket — fires automatically on every key game action.
- Fires on **discard, peng, gang, win**. Commentary reaches the browser within the target latency on Gemini Flash Lite.
- **History replay:** viewers joining mid-game instantly receive the last 50 commentary messages — no gap in the narrative.
- **Multi-room:** concurrent game rooms each run their own isolated commentary pipeline — rooms never interfere.

Connected to a real Mahjong server — not a simulation.

Narrative

Context-Aware Commentary

- Unlike naive LLM commentary, the AI **tracks the full game arc** — it knows what happened 30 moves ago, not just the current tile.
- The background Summarizer writes a **2–3 sentence narrative** from the complete event history every 8 moves, kept fresh without ever blocking the live commentary.
- **Objective hand-strength** signal (shanten number, computed by algorithm) fed into every prompt — LLM doesn't guess at strategic significance.
- TTS audio via OpenAI: commentary is spoken aloud, queued so **speech never overlaps** across rapid events.

No fine-tuning, no training data — pure prompt engineering and structured context.

Constant

Stable, Scalable Architecture

- **Fixed prompt size** by design — system prompt + narrative (~60 tok) + 10-move window (~150 tok) + current action (~80 tok). Move 80 costs the same as move 8.
- **Debug sidebar:** player hands as tile chips, live shanten number, scrollable event trace — AI reasoning is fully transparent and inspectable.
- **Zero game modification:** integrates as a WebSocket listener — no engine changes, no SDK, no source access required.
- **React web UI** — any viewer joins a room by URL; commentary and audio begin immediately on connect.

Integrates with any Mahjong server as a zero-touch WebSocket listener.

Thank You

The Concept | The Approach | The Assumptions | The Dataset | The Results

Youlin Qu · Lenghan Zhu · Rongcheng Guo · Pingfan Bu